TextFlappyBird: GLIE MC Control & SARSA(λ)

Adonis JAMAL[1,2]

[1] CentraleSupélec, MDS-MMF
[2] École Normale Supérieure Paris-Saclay, MVA

Abstract. We implement and compare two tabular reinforcement learning agents on the Text Flappy Bird environment: a first-visit GLIE Monte-Carlo control agent and a SARSA(λ) agent with accumulating eligibility traces. Hyperparameters are selected via grid search. SARSA(λ) achieves perfect greedy play, while MC learns a reasonable but high-variance policy. A generalization study reveals that both agents degrade sharply on harder configurations. Code: github.com/adonis107/MDS-RL.

Keywords: Reinforcement Learning · Monte-Carlo Control · SARSA(λ)

1 Introduction

Text Flappy Bird (TFB) [1] is a text-based Gymnasium environment where the agent controls a bird navigating through pipe gaps by choosing to flap or idle ($a \in \{0, 1\}$). We use `TextFlappyBird-v0`, which returns a compact state $(x_{\text{dist}}, y_{\text{dist}})$ representing the distance to the nearest pipe gap centre. This low-dimensional representation makes tabular methods directly applicable. We implement two agents: a first-visit GLIE Monte-Carlo (MC) control agent and a SARSA(λ) agent with accumulating eligibility traces [3].

2 Methods

All experiments use `TextFlappyBird-v0` with training configuration `height=15`, `width=20`, `pipe_gap=4`, $\gamma = 0.99$. The reward is $+1$ per survived step; greedy evaluation episodes are capped at 5 000 steps.

GLIE Monte-Carlo Control. The agent maintains a Q-table updated via the incremental-mean first-visit rule:

$$Q(s, a) \leftarrow Q(s, a) + \frac{1}{N(s, a)}\big(G_t - Q(s, a)\big). \tag{1}$$

Three epsilon-decay schedules are considered: *linear* ($\epsilon \leftarrow \epsilon_{\max} - k\delta$), *inverse* ($\epsilon = 1/k$), and *exponential* ($\epsilon = \epsilon_{\min} + (1 - \epsilon_{\min})e^{-\lambda k}$), where k is the episode number. Strictly, GLIE requires $\epsilon_k \to 0$; only the inverse schedule satisfies this exactly. Our best configuration uses exponential decay with $\epsilon_{\min} = 0.05 > 0$, which relaxes GLIE to ensure continued exploration. Convergence is therefore not theoretically guaranteed, though the agent still learns an effective policy in practice.

SARSA(λ). The agent performs online TD updates with accumulating eligibility traces (Section 12.7 of [3]):

$$\delta_t = R_{t+1} + \gamma\, Q(S', A') - Q(S, A), \quad E(S, A) \leftarrow E(S, A) + 1, \tag{2}$$

$$\forall (s, a): \quad Q(s, a) \leftarrow Q(s, a) + \alpha\, \delta_t\, E(s, a), \quad E(s, a) \leftarrow \gamma\, \lambda\, E(s, a). \tag{3}$$

Traces below 10^{-6} are pruned. Exploration follows the inverse schedule $\epsilon = \max(\epsilon_{\min}, 1/k)$.

Grid Search. Hyperparameters are selected via parallel grid search (10 000 training episodes, 500 greedy evaluation episodes). For MC: decay type, decay parameter, and $\epsilon_{\min} \in \{0.01, 0.05\}$ (10 configs). For SARSA: $\alpha \in \{0.05, 0.1, 0.2\}$, $\lambda \in \{0.6, 0.8, 0.9\}$, and $\epsilon_{\min} \in \{0.01, 0.05\}$ (18 configs). The best configuration is then trained for 50 000 episodes.

3 Results

Grid Search. The MC grid search (Fig. 1) selects exponential decay ($\lambda = 0.0005$, $\epsilon_{\min} = 0.05$). Exponential decay dominates; fast linear decay (step = 0.005) performs poorly as exploration ends prematurely.

For SARSA (Fig. 2), the best configuration is $\alpha = 0.1$, $\lambda = 0.6$, $\epsilon_{\min} = 0.05$. Higher α values (0.1–0.2) consistently outperform lower ones at $\epsilon_{\min} = 0.05$, and $\alpha = 0.1$ provides the best speed-stability balance.

Training and Evaluation. Greedy evaluation over 1 000 episodes after full training is reported in Table 1. SARSA(λ) achieves perfect greedy play, while MC has high variance with frequent early failures (Fig. 5). The training curves (Figs. 3, 4) show SARSA converging faster and more smoothly (average training reward $\sim$200 vs. $\sim$120 for MC), as eligibility traces propagate credit within each episode rather than waiting for termination. Training-time rewards are lower than greedy evaluation because exploration ($\epsilon > 0$) forces suboptimal actions.

State-Value Functions. Fig. 6 shows $V(s) = \max_a Q(s, a)$ for each agent. Both assign the highest values near the gap centre ($y_{\text{dist}} \approx 0$), with lower values at extreme offsets. SARSA's value surface is smooth and uniformly high in the safe region. MC exhibits low-value pockets, particularly at moderate x_{dist} with $|y_{\text{dist}}| \geq 2$: these states are visited infrequently yet appear early in long episodes, so their Q-values receive few first-visit updates, each based on high-variance full returns. This directly explains MC's occasional greedy failures (min reward = 18 in Table 1): encountering a poorly estimated state triggers a suboptimal action cascade ending in a crash.

Configuration Generalization. Both agents are evaluated on modified configurations, varying one parameter at a time (Fig. 7). **Pipe gap** has the strongest effect: near-perfect scores for gap ≥ 4 but collapse at gap = 2 (MC: 14, SARSA: 31). **Height/Width** beyond training values also degrades performance, as novel states are unseen during training. The 2D heatmap (Fig. 8) confirms difficulty rises steeply as pipe gap shrinks and height increases.

4 Discussion

Agent Comparison. MC updates Q-values only at episode end using full returns: states visited early in long episodes receive delayed, high-variance corrections, explaining the low-value pockets in Fig. 6 and greedy failures (min = 18). SARSA(λ) updates online at every step via eligibility traces, so even states encountered once mid-episode receive immediate credit, yielding a smoother value surface and zero-variance greedy play. The trade-off is that SARSA introduces hyperparameters (α, λ); in particular, higher α (0.1–0.2) performs best because ongoing ϵ-greedy exploration injects noise into returns, and a larger step-size allows the agent to correct for this noise more quickly.

Environment Versions and Transferability. `TextFlappyBird-v0` returns a compact 2D state $(x_{\text{dist}}, y_{\text{dist}})$, yielding $\sim$1 000 tabular entries. `TextFlappyBird-screen-v0` returns the full 20×15 character grid, producing a combinatorially vast state space requiring function approximation. The simple variant discards information beyond the nearest pipe (e.g., upcoming gaps, velocity). Transferring to the pixel-based Flappy Bird [4] would require a DQN-style pipeline [2]: stacking consecutive frames to capture velocity, a convolutional encoder to extract spatial features, an experience replay buffer for sample efficiency, and a target network to stabilise bootstrapped updates.

Configuration Generalization. Both agents generalise to easier configurations but fail on harder ones (Fig. 7), as tabular methods cannot interpolate to unseen states. Training on a harder configuration could improve robustness at the cost of longer training.

Limitations. The 5 000-step evaluation cap means SARSA's apparent perfection may mask subtle policy flaws visible only over longer horizons. Grid search with 10 000 episodes is a rough proxy: the ranking may shift at 50k. Both agents rely on exact state matching with no interpolation, so even minor environment changes introduce unseen states.

5 Conclusion

SARSA(λ) learns an optimal policy for TFB, achieving perfect greedy play thanks to online trace-based credit assignment. MC learns a functional but unreliable policy (mean reward 1 860 vs. the 5 000-step cap), as its episode-end batch updates leave many state-action values insufficiently corrected. Neither agent generalises to harder configurations, highlighting the fundamental limitation of tabular methods when the state space varies across settings.

References

1. Christodoulidis, S.: Text flappy bird gym. `https://gitlab-research.centralesupelec.fr/stergios.christodoulidis/text-flappy-bird-gym` (2021)

2. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M.A., Fidjeland, A.K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D.: Human-level control through deep reinforcement learning. Nature **518**, 529–533 (2015), `https://api.semanticscholar.org/CorpusID:205242740`
3. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. MIT Press, 2nd edn. (2018)
4. Talendar: Flappy bird gym. `https://github.com/Talendar/flappy-bird-gym` (2020)

Appendix

Table 1. Greedy evaluation after 50 000 training episodes.

Metric	MC	SARSA(λ)
Mean reward	1 860	5 000
Std reward	1 556	0
Min / Max	18 / 5 000	5 000 / 5 000
Q-table entries	1 038	1 066

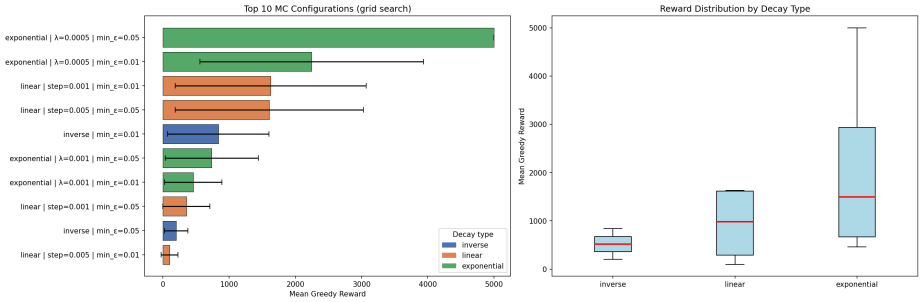

Fig. 1. MC grid search results. Left: top configurations by mean greedy reward. Right: reward distribution by decay type.

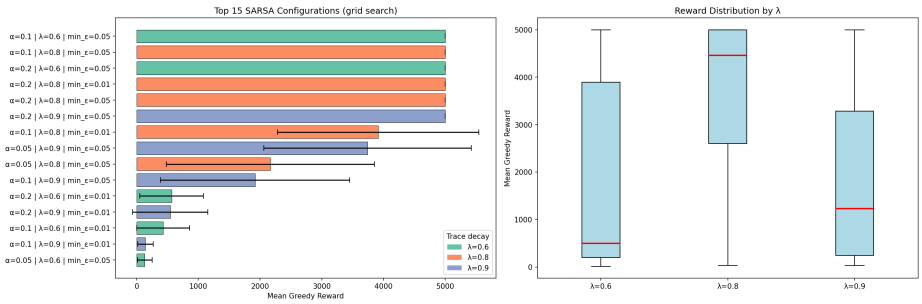

Fig. 2. SARSA(λ) grid search results. Left: top configurations by mean greedy reward. Right: reward distribution by trace decay λ.

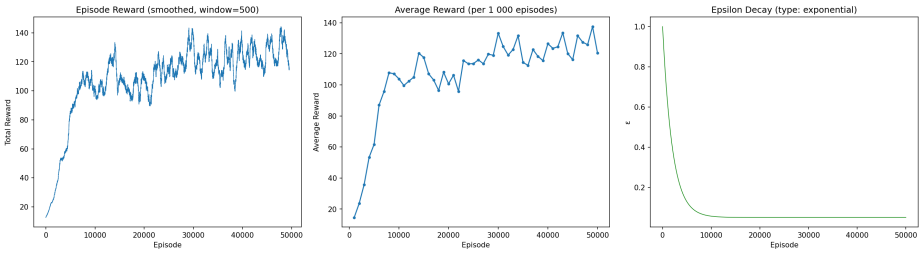

Fig. 3. MC training curves over 50 000 episodes.

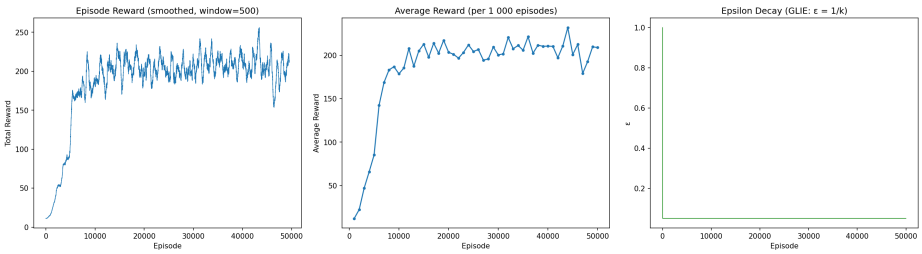

Fig. 4. SARSA(λ) training curves over 50 000 episodes.

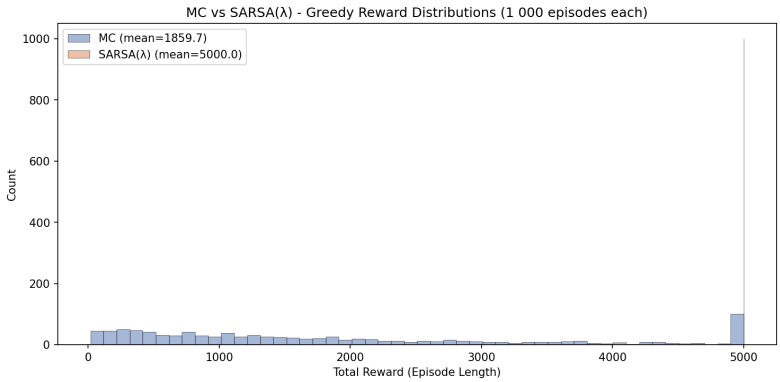

Fig. 5. Overlaid greedy reward distributions (1 000 episodes each).

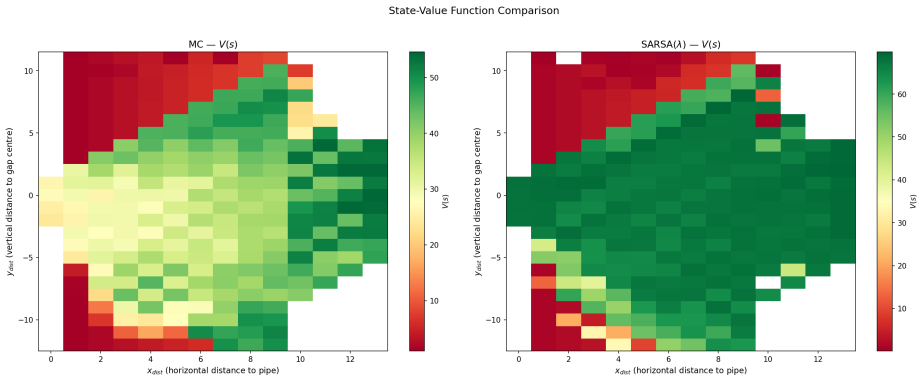

Fig. 6. State-value function $V(s) = \max_a Q(s, a)$ for both agents. States near the gap centre ($y_{\text{dist}} \approx 0$) have the highest values.

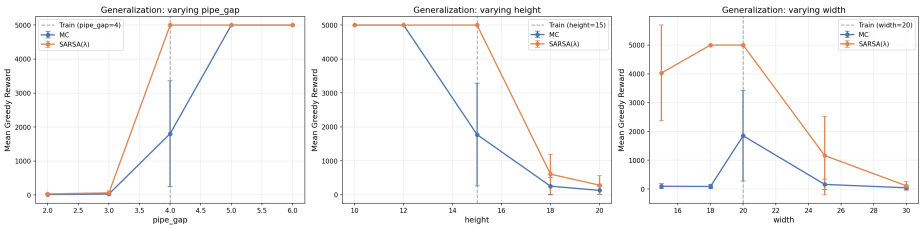

Fig. 7. Mean greedy reward when varying one environment parameter. Dashed line: training value.

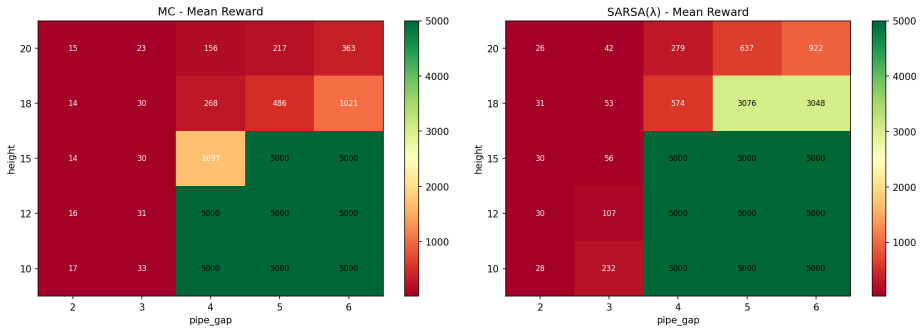

Fig. 8. 2D heatmap of mean greedy reward across pipe gap and height (width fixed at 20).